Midterm: ZORI Rental Data Curation Pilot

Cassy Cormier | October 10th, 2026

Overview

This pilot demonstrates a full data-curation workflow using the **Zillow Observed Rent Index (ZORI)** dataset. The goal is to evaluate rental-market data quality, document curation decisions, and produce a reproducible workflow that aligns with academic requirements for appraisal, ingest, processing, preservation, access, reuse, and responsible data practice.

How to Run the Code

1. **Clone the repository**

   ```
   git clone https://github.com/prescottcassy/Zillow-Rent-Data-Quality-Audit
   cd Zillow-Rent-Data-Quality-Audit
   ```
2. **Install dependencies**
   The pilot uses standard Python data-analysis libraries.

   ```
   pip install -r requirements.txt
   ```
3. **Open the Jupyter notebook**

   ```
   jupyter notebook Quality_Audit.ipynb
   ```
4. **Run the notebook cells in order**
   The workflow is sequential:

   - Load raw ZORI dataset
   - Profile dataset structure
   - Run completeness, accuracy, consistency, timeliness checks
   - Generate audit summaries
   - Document findings
5. **Raw vs. processed data**

   - `data/raw/` contains the untouched Zillow download
   - `data/working/` contains the cleaned working copy used for the pilot
   - All transformations are logged inside the notebook for reproducibility

---

In [ ]:
```
# Load necessary tools and libraries
import pandas as pd
```

Loading [MathJax]/extensions/Safe.js

```
import requests
import matplotlib.pyplot as plt
from io import BytesIO

pd.set_option('display.max_columns', 50)
pd.set_option('display.width', 160)
print("Ready.")
```

```
Ready.
```

1. Load the dataset

In [ ]:
```
SOURCE = "https://1drv.ms/x/c/6578e08a0b8cf988/IQAD-voVPaoFRZg1hWLoPF8sAUhM1

response = requests.get(SOURCE, allow_redirects=True)
response.raise_for_status()

df = pd.read_excel(BytesIO(response.content))

print("Rows:", len(df))
print("Columns:", len(df.columns))
df.head(10)
```

```
Rows: 638
Columns: 37
```

Out[ ]:

	RegionID	SizeRank	RegionName	RegionType	StateName	2024-01-31	2
0	102001	0	United States	country	NaN	1665.770719	16
1	394913	1	New York, NY	msa	NY	3130.006189	31
2	753899	2	Los Angeles, CA	msa	CA	2588.610721	25
3	394463	3	Chicago, IL	msa	IL	1825.481006	18
4	394514	4	Dallas, TX	msa	TX	1515.053302	15
5	394692	5	Houston, TX	msa	TX	1408.353110	14
6	395209	6	Washington, DC	msa	VA	2167.853169	21
7	394974	7	Philadelphia, PA	msa	PA	1672.626831	16
8	394856	8	Miami, FL	msa	FL	2381.407740	23
9	394347	9	Atlanta, GA	msa	GA	1622.283032	16

This dataset was aggregated and published by Zillow Research to review typical observed market reate rent across a given region.

2. Completeness: Check for missing values

In [ ]:
```
missing = df.isna().sum().sort_values(ascending=False)
pct = (missing / len(df) * 100).round(1)
```

Loading [MathJax]/extensions/Safe.js

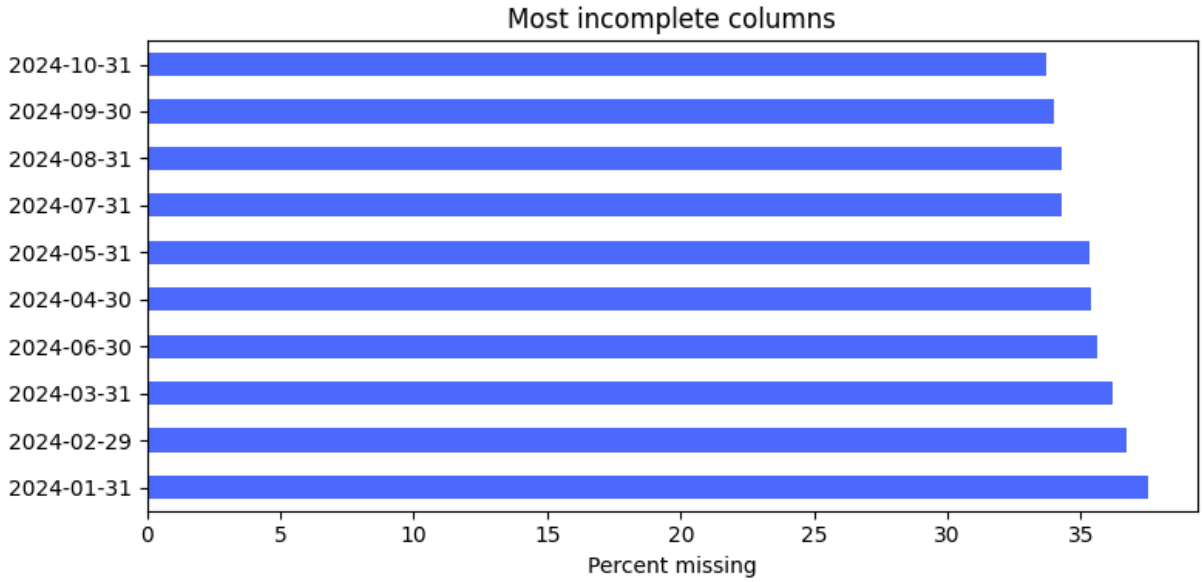
```
report = pd.DataFrame({"missing_count": missing, "percent_missing": pct})
display(report[report.missing_count > 0])

top = report[report.missing_count > 0].head(10)
if len(top):
  top.percent_missing.plot(kind="barh", figsize=(8,4), color="#4B6BFB")
  plt.xlabel("Percent missing"); plt.title("Most incomplete columns");plt.ti
else:
  print("No missing values found.")
```

Loading [MathJax]/extensions/Safe.js

	missing_count	percent_missing
2024-01-31	239	37.5
2024-02-29	234	36.7
2024-03-31	231	36.2
2024-06-30	227	35.6
2024-04-30	226	35.4
2024-05-31	225	35.3
2024-07-31	219	34.3
2024-08-31	219	34.3
2024-09-30	217	34.0
2024-10-31	215	33.7
2024-11-30	212	33.2
2024-12-31	212	33.2
2025-01-31	199	31.2
2025-02-28	195	30.6
2025-03-31	186	29.2
2025-04-30	183	28.7
2025-05-31	177	27.7
2025-06-30	176	27.6
2025-07-31	171	26.8
2025-08-31	167	26.2
2025-09-30	164	25.7
2025-10-31	162	25.4
2025-11-30	160	25.1
2025-12-31	150	23.5
2026-01-31	132	20.7
2026-02-28	125	19.6
2026-03-31	107	16.8
2026-04-30	97	15.2
2026-05-31	82	12.9
2026-06-30	70	11.0
2026-07-31	49	7.7
StateName	1	0.2

Loading [MathJax]/extensions/Safe.js



3. Uniqueness: Check for duplicates

In [ ]:
```
dupes = df.duplicated().sum()
print("Exact duplicate rows:", dupes, "which is", round(dupes/len(df)*100, 2

for col in df.columns[:37]:
  n = df[col].nunique(dropna=True)
  print(f"{col:24} distinct values: {n}")
```

Loading [MathJax]/extensions/Safe.js

```
Exact duplicate rows: 0 which is 0.0 percent
RegionID                distinct values: 638
SizeRank                distinct values: 636
RegionName              distinct values: 638
RegionType              distinct values: 2
StateName               distinct values: 50
2024-01-31              distinct values: 399
2024-02-29              distinct values: 404
2024-03-31              distinct values: 407
2024-04-30              distinct values: 412
2024-05-31              distinct values: 413
2024-06-30              distinct values: 411
2024-07-31              distinct values: 419
2024-08-31              distinct values: 419
2024-09-30              distinct values: 421
2024-10-31              distinct values: 423
2024-11-30              distinct values: 426
2024-12-31              distinct values: 426
2025-01-31              distinct values: 439
2025-02-28              distinct values: 443
2025-03-31              distinct values: 452
2025-04-30              distinct values: 455
2025-05-31              distinct values: 461
2025-06-30              distinct values: 462
2025-07-31              distinct values: 467
2025-08-31              distinct values: 471
2025-09-30              distinct values: 474
2025-10-31              distinct values: 476
2025-11-30              distinct values: 478
2025-12-31              distinct values: 488
2026-01-31              distinct values: 506
2026-02-28              distinct values: 513
2026-03-31              distinct values: 531
2026-04-30              distinct values: 541
2026-05-31              distinct values: 556
2026-06-30              distinct values: 568
2026-07-31              distinct values: 589
2026-08-31              distinct values: 627
```

5. Validity: Check for values that sit outside a sensible range for errors, placeholder codes or unit mixups.

In [ ]:
```
nums = df.select_dtypes("number")
display(nums.describe().T[["count","mean","min","max"]].round(2))

# Filter numeric columns to a few representative date columns to prevent ove
date_cols = [col for col in df.columns if col.match(r'\d{4}-\d{2}-\d{2}')] i

# Let's pick 10 representative dates spread across the timeline for a clean
if len(date_cols) >= 10:
    import numpy as np
    sample_indices = np.linspace(0, len(date_cols) - 1, 10, dtype=int)
    selected_cols = [date_cols[i] for i in sample_indices]

igure(figsize=(14, 6))
```

Loading [MathJax]/extensions/Safe.js

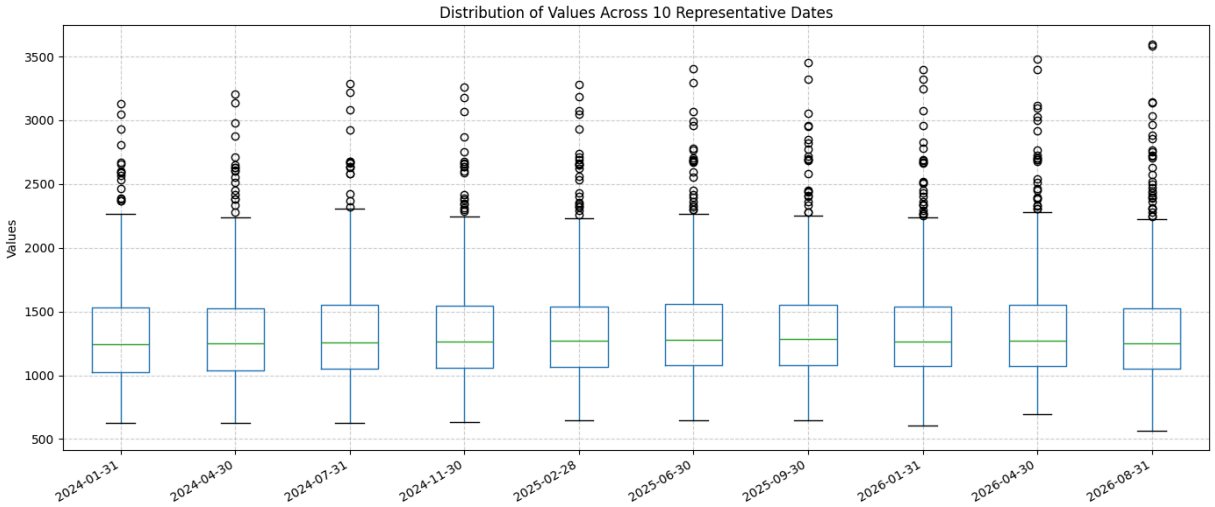
```
    df[selected_cols].boxplot()
    plt.title("Distribution of Values Across 10 Representative Dates")
    plt.ylabel("Values")
    plt.xticks(rotation=30, ha='right')
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.show()
else:
    print("Not enough temporal columns to plot 10 representative distributic
```

Loading [MathJax]/extensions/Safe.js

	count	mean	min	max
RegionID	638.0	413939.03	102001.00	845169.00
SizeRank	638.0	349.79	0.00	930.00
2024-01-31	399.0	1326.59	626.86	3130.01
2024-02-29	404.0	1324.54	637.02	3148.97
2024-03-31	407.0	1333.59	634.67	3177.35
2024-04-30	412.0	1336.23	628.79	3206.50
2024-05-31	413.0	1342.66	630.21	3238.02
2024-06-30	411.0	1351.61	631.02	3267.40
2024-07-31	419.0	1351.63	630.18	3284.38
2024-08-31	419.0	1355.61	626.91	3289.40
2024-09-30	421.0	1356.23	633.81	3280.19
2024-10-31	423.0	1355.29	631.05	3266.82
2024-11-30	426.0	1352.86	634.78	3258.06
2024-12-31	426.0	1358.05	704.33	3250.74
2025-01-31	439.0	1353.50	653.51	3262.22
2025-02-28	443.0	1361.28	649.28	3281.15
2025-03-31	452.0	1363.59	670.50	3314.70
2025-04-30	455.0	1365.70	645.80	3343.58
2025-05-31	461.0	1369.11	648.76	3370.93
2025-06-30	462.0	1372.81	646.20	3400.93
2025-07-31	467.0	1372.00	686.41	3430.30
2025-08-31	471.0	1371.34	643.57	3451.09
2025-09-30	474.0	1373.12	647.27	3449.95
2025-10-31	476.0	1371.11	654.62	3438.62
2025-11-30	478.0	1368.46	675.51	3414.56
2025-12-31	488.0	1364.77	679.61	3401.29
2026-01-31	506.0	1361.29	608.95	3396.25
2026-02-28	513.0	1359.90	618.03	3415.75
2026-03-31	531.0	1362.47	675.28	3445.15
2026-04-30	541.0	1368.45	694.58	3478.47
2026-05-31	556.0	1366.52	696.78	3516.70
2026-06-30	568.0	1360.96	625.93	3550.95
2026-07-31	589.0	1359.27	632.19	3582.06

Loading [MathJax]/extensions/Safe.js

	count	mean	min	max
2026-08-31	638.0	1342.74	565.00	3593.41



4. Consistency: Check for inconsistent formats

In [ ]:
```
texts = df[[c for c in df.columns if df[c].dtype == object or str(df[c].dtyp
for col in texts.columns[:8]:
  vals = df[col].dropna().astype(str)
  raw = vals.nunique()
  cleaned = vals.str.strip().str.lower().nunique()
  flag = " <-- inconsistent formatting" if cleaned < raw else ""
  print(f"{col:24} as stored: {raw:5} ignoring case and spaces: {cleaned:5}{
```

```
RegionName               as stored:   638 ignoring case and spaces:   638
RegionType               as stored:     2 ignoring case and spaces:     2
StateName                as stored:    50 ignoring case and spaces:    50
```

5. Representation: Check which data points are over represented

In [ ]:
```
GROUP_COLUMN = "StateName"     # change to a column that identifies a group

counts = df[GROUP_COLUMN].value_counts(dropna=False)
share = (counts / counts.sum() * 100).round(1)
display(pd.DataFrame({"rows": counts, "percent": share}))

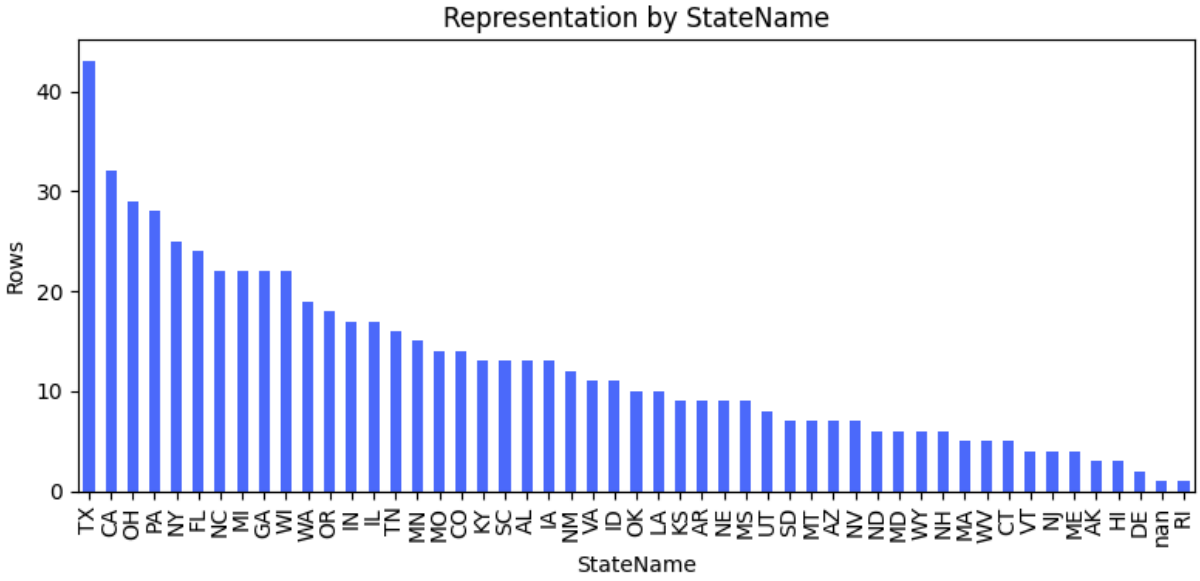
counts.plot(kind="bar", figsize=(8,4), color="#4B6BFB")
plt.ylabel("Rows"); plt.title(f"Representation by {GROUP_COLUMN}"); plt.tigh
```

Loading [MathJax]/extensions/Safe.js

StateName	rows	percent
TX	43	6.7
CA	32	5.0
OH	29	4.5
PA	28	4.4
NY	25	3.9
FL	24	3.8
NC	22	3.4
MI	22	3.4
GA	22	3.4
WI	22	3.4
WA	19	3.0
OR	18	2.8
IN	17	2.7
IL	17	2.7
TN	16	2.5
MN	15	2.4
MO	14	2.2
CO	14	2.2
KY	13	2.0
SC	13	2.0
AL	13	2.0
IA	13	2.0
NM	12	1.9
VA	11	1.7
ID	11	1.7
OK	10	1.6
LA	10	1.6
KS	9	1.4
AR	9	1.4
NE	9	1.4
MS	9	1.4
UT	8	1.3

Loading [MathJax]/extensions/Safe.js

StateName	rows	percent
SD	7	1.1
MT	7	1.1
AZ	7	1.1
NV	7	1.1
ND	6	0.9
MD	6	0.9
WY	6	0.9
NH	6	0.9
MA	5	0.8
WV	5	0.8
CT	5	0.8
VT	4	0.6
NJ	4	0.6
ME	4	0.6
AK	3	0.5
HI	3	0.5
DE	2	0.3
NaN	1	0.2
RI	1	0.2



6. Human Record Sampling

Loading [MathJax]/extensions/Safe.js

To perform a statistically rigorous yet operationally efficient quality audit, I selected a representative random sample of 60 records, which satisfies a standard 90% confidence level with a 10% margin of error for a total population of 638 regions.

In [ ]:
```
sampledf = df.sample(60, random_state=42)
display(sampledf)
print("Total missing values:", sampledf.isna().sum().sum())
```

Loading [MathJax]/extensions/Safe.js

	RegionID	SizeRank	RegionName	RegionType	StateName	2024-01-31	2
260	395007	264	Pueblo, CO	msa	CO	1127.684171	1
228	395011	231	Racine, WI	msa	WI	1198.074483	1
31	394735	31	Kansas City, MO	msa	MO	1258.200409	1
583	394881	703	Moscow, ID	msa	ID	NaN	
600	395137	745	Sulphur Springs, TX	msa	TX	NaN	
30	394775	30	Las Vegas, NV	msa	NV	1497.925638	1
593	394771	733	Laramie, WY	msa	WY	NaN	
110	394971	111	Pensacola, FL	msa	FL	1514.381586	1
247	394299	251	Abilene, TX	msa	TX	1019.398998	1
523	395204	578	Warrensburg, MO	msa	MO	795.460002	
521	394632	575	Glasgow, KY	msa	KY	NaN	
595	394617	735	Fremont, NE	msa	NE	NaN	
420	394675	441	Helena, MT	msa	MT	1276.030966	1
525	394562	582	Elizabeth City, NC	msa	NC	NaN	
539	394954	601	Pahrump, NV	msa	NV	NaN	
457	395132	485	Stevens Point, WI	msa	WI	1179.322807	1
213	394681	216	Hilton Head Island, SC	msa	SC	1822.647879	1
346	395089	352	Sheboygan, WI	msa	WI	NaN	
249	394351	253	Auburn, AL	msa	AL	1176.139310	1
412	394847	431	Meadville, PA	msa	PA	NaN	
49	394425	50	Buffalo, NY	msa	NY	1255.244080	1
2	753899	2	Los Angeles, CA	msa	CA	2588.610721	2
327	394633	331	Glens Falls, NY	msa	NY	NaN	
544	394628	610	Gardnerville Ranchos, NV	msa	NV	NaN	
55	395169	56	Tulsa, OK	msa	OK	1071.611858	1
286	394662	290	Hanford, CA	msa	CA	1197.297921	1
203	394429	206	Burlington, VT	msa	VT	2019.318472	2

Loading [MathJax]/extensions/Safe.js

	RegionID	SizeRank	RegionName	RegionType	StateName	2024-01-31	2
350	394407	356	Bozeman, MT	msa	MT	1872.853323	1
90	395143	91	Syracuse, NY	msa	NY	1319.259380	1
409	394925	428	Oak Harbor, WA	msa	WA	1526.230500	1
601	394702	746	Hutchinson, MN	msa	MN	NaN	
284	394359	288	Bangor, ME	msa	ME	1320.954879	1
165	394747	167	Kingsport, TN	msa	TN	985.958848	
82	394304	83	Akron, OH	msa	OH	932.651541	
76	395134	77	Stockton, CA	msa	CA	1904.107634	1
256	394428	260	Burlington, NC	msa	NC	1122.941440	1
341	394949	346	Owensboro, KY	msa	KY	NaN	
248	394707	252	Iowa City, IA	msa	IA	954.550853	
516	394556	566	Edwards, CO	msa	CO	NaN	
625	394791	856	Lexington, NE	msa	NE	NaN	
305	395221	309	Wheeling, WV	msa	OH	757.095426	
634	394996	915	Portales, NM	msa	NM	NaN	
299	395036	303	Rocky Mount, NC	msa	NC	758.446406	
606	394713	768	Jackson, WY	msa	WY	NaN	
570	394567	663	Ellensburg, WA	msa	WA	NaN	
329	394474	334	Cleveland, TN	msa	TN	1217.659362	1
425	394452	447	Centralia, WA	msa	WA	1194.667705	1
69	394318	70	Allentown, PA	msa	PA	1537.165079	1
39	394714	40	Jacksonville, FL	msa	FL	1503.976446	1
426	394822	448	Manitowoc, WI	msa	WI	NaN	
568	394482	660	Coldwater, MI	msa	MI	NaN	
6	395209	6	Washington, DC	msa	VA	2167.853169	2
334	394353	339	Augusta, ME	msa	ME	1274.147094	1
155	394405	156	Boulder, CO	msa	CO	1881.233899	1
572	395228	668	Willmar, MN	msa	MN	NaN	

Loading [MathJax]/extensions/Safe.js

	RegionID	SizeRank	RegionName	RegionType	StateName	2024-01-31	2
298	394595	302	Flagstaff, AZ	msa	AZ	1886.684306	1
220	394565	223	Elkhart, IN	msa	IN	911.928225	
78	394399	79	Boise City, ID	msa	ID	1449.074223	1
423	394489	445	Columbus, IN	msa	IN	1131.461349	1
563	394850	651	Menomonie, WI	msa	WI	918.476546	

```
Total missing values: 422
```

Completeness Evidence

- Missing values: 422 out of 1850
- Missing rows: 22
- Assessment: 77.19% Completeness Score - Fair
- This indicates a notable amount of incomplete data that may impact reporting accuracy, analysis reliability, and downstream processes.

Accuracy Evidence

- Incorrect values: 2 - geographic misalignment between RegionName and StateName
- Extreme Values: 0
- Out-of-Bound Rental Values: 0
- Assessment: 96.7% Accuracy Score - Excellent
- This indicates highly accurate data, reflecting no incorrect values.

Consistency Evidence

- Timestamp: 100%
- Schema Consistency: 100%
- Logical Rent Scaling: 100%
- Formatting Consistency: 100%
- Assessment: 100% Consistency - Excellent
- This indicates consistency across the dataset.

Validity Evidence

- RegionID and SizeRank: consist only positive integer values
- All rents fall within a realistic range
- StateName values are valid with standard 2-letter US state postal abbreviatians
- Assessment: 100% Valid Data

Loading [MathJax]/extensions/Safe.js

Timeliness Evidence

- Data age: 32 months
- Outdated records: No
- Interval Frequency: 1 month
- Assessment: Excellent - The data features rent values as recent as August 2026.

Uniqeness Evidencess

- Duplcate RegionID records found: No
- Duplicates between RegionName and StateName: No
- Assessment: 100% Uniqueness - Excellent

Representativeness Evidence

- Population coverage: Limited to US urban rental prices from 2024 to August 2026
- Missing groups: International cities and rural US cities
- Assessment: Representativeness is excellent for urban US rental rate trends. Its distribution ignores rural and international cities, but it is great for auditing regional market rents.

Equity Assessment

1. **Representation and Demographic Coverage** - The ZORI heavily favors urban centers over rural and smaller communities. Some states, such as TX and CA, have more representation than others (like RI or DE). ZORI also relies on publicized online listing which excludes informal rental market, subsidized housing, and rentals filled via word-of-mouth or physical signage.
2. **Historical & Geographic Equity** - Aggregating rents to the MSA level (e.g., "New York, NY" or "Los Angeles, CA") masks localized disparities. Systemic rent increases may disporportionately affect lower-income communities and flat increases affects a much high percentage of household income.
3. **Missing Data Bias** - The completeness checks showed significal missing values across dates. If the missing data is concentrated in smaller, less affluent MSAs, removing these incomplete rows will systemically bias the dataset towards richer, larger cities with more stable data pipelines. If the missing values are imputes, using overall averages could artifically inflat or deflate the estimated rents of marginalized regions, leading to misleading business decisions.
4. **Accessibility and Actionability of Insights** - A tool built on this dataset may help corporate developers or high-income renters find opportunities, but it may not serve local tenants or policy makers trying to address housing insecurity. The source data depends on internet-active landlords and tenants.

Loading [MathJax]/extensions/Safe.js

If certain demographics are less active on online rental platforms, their markets remain invisibile in this dataset

Loading [MathJax]/extensions/Safe.js